

Additional task for Today (12/4 Thursday 2025)

1. Install any vector store on your machine
2. Install Ollama and get mistral 8 B param model
3. Upload new contracts using RAG and do question-answering on mistral on your machine
4. You do QA and as well as “Create Contract”

Details of the Feature to be built for “Generate Contract” (Same as create contract

1. Menu: Below Analytics : “Generate Contract”
2. This functionality should leverage LLM with RAG (LLM brings general purpose language and RAG brings context)
3. User is allowed to upload say (5 contracts of a particular type), in ms-word or pdf format
4. LLM + RAG understands the contract
5. Query like window allows use to enter text e.g. “ Generate contract similar to the uploaded contracts and make following changes 1. First Party PrimeNumerics, Second Party: Infosys 2. Duration of contract : date1 to date 2 3 and so on.” User can give more detailed prompt
6. System should generate the contract using LLM+RAG

This guide outlines the steps to build the Agentic AI environment on **Ubuntu 24.04** using native binaries and package managers, removing the dependency on Docker.

1. System Preparation

Update your system and install essential build tools required for Python and Node.js extensions.

Bash

```
# Update Ubuntu repositories
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install common dependencies
```

```
sudo apt install curl git build-essential libssl-dev zlib1g-dev unzip -y
```

2. Install & Run Qdrant (Native Binary)

Since we are not using Docker, we will download the pre-compiled binary from Qdrant's GitHub repository and run it directly.

2.1 Download Qdrant

Bash

```
# Create a directory for Qdrant
```

```
mkdir -p ~/qdrant_home/storage
```

```
cd ~/qdrant_home
```

```
# Download the latest Linux binary (check Qdrant GitHub releases for newer versions)
```

```
wget
```

```
https://github.com/qdrant/qdrant/releases/download/v1.10.1/qdrant-x86\_64-unknown-linux-gnu.tar.gz
```

```
# Extract the tarball
```

```
tar -xzf qdrant-x86_64-unknown-linux-gnu.tar.gz
```

2.2 Run Qdrant You can run it directly in your terminal. For long-term execution, use **tmux** or create a systemd service.

Bash

```
# Allow execution
chmod +x qdrant
```

```
# Run Qdrant (Defaults: HTTP port 6333, gRPC port 6334)
./qdrant
```

- **Keep this terminal open** or run it in a background session (`nohup ./qdrant &`).
 - **Verification:** Visit <http://localhost:6333/dashboard> in your browser.
-

3. Install Ollama (Local LLM Server)

Ollama provides a native install script that works perfectly on Ubuntu without containers.

3.1 Install Ollama

```
Bash
curl -fsSL https://ollama.com/install.sh | sh
```

3.2 Start & Pull Models Ollama usually starts automatically as a system service.

```
Bash
# Check if running
sudo systemctl status ollama
```

```
# Pull the Mistral model (optimized for agentic tasks)
ollama pull mistral
```

- **Endpoint:** Your LLM is now accessible at <http://localhost:11434>.
-

4. Install Redis (State Persistence)

LangGraph needs a persistence layer to save conversation history (checkpoints). On a non-Docker setup, a native Redis installation is the standard choice.

```
Bash
# Install Redis Server
sudo apt install redis-server -y
```

```
# Enable and start the service
sudo systemctl enable --now redis-server

# Verify connection (should reply with "PONG")
redis-cli ping
```

5. Python Backend (LangGraph + CrewAI)

Set up the Python environment using `venv`.

5.1 Setup Virtual Environment

```
Bash
# Create backend directory
mkdir -p ~/agentic-ai/backend
cd ~/agentic-ai/backend

# Install Python venv
sudo apt install python3.12-venv -y

# Create and activate environment
python3 -m venv venv
source venv/bin/activate
```

5.2 Install Python Dependencies

```
Bash
# Upgrade pip
pip install --upgrade pip

# Install Core Frameworks
pip install langgraph crewai crewai[tools] langchain langchain-community

# Install Integrations
pip install langchain-qdrant langchain-ollama redis

# Install API & Server tools
pip install fastapi uvicorn websockets
```

6. Node.js & React Frontend

Use **nvm** (Node Version Manager) to avoid permission issues.

6.1 Install Node.js

Bash

Install NVM

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Load NVM (run this or restart terminal)

```
export NVM_DIR="$HOME/.nvm"
```

```
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
```

Install Node.js

```
nvm install 20
```

```
nvm use 20
```

6.2 Setup React Project

Bash

```
cd ~/agentic-ai
```

```
npx create-next-app@latest frontend
```

Select: Yes to TypeScript, Tailwind, ESLint, App Router.

6.3 Install Frontend Client Libraries

Bash

```
cd frontend
```

```
npm install ai @langchain/langgraph-sdk socket.io-client
```

7. Final Verification

To ensure your non-Docker stack is communicating correctly, create a simple Python check script in `~/agentic-ai/backend/check_stack.py`:

Python

```
import requests
```

```
from qdrant_client import QdrantClient
```

```
from langchain_ollama import ChatOllama
```

```
def check_services():
```

```
    # 1. Check Ollama
```

```
try:
    llm = ChatOllama(model="mistral")
    print("✅ Ollama: Connected (Response: " + llm.invoke("Hi").content[:10] + "...)")
except Exception as e:
    print(f"❌ Ollama: Failed - {e}")
```

2. Check Qdrant

```
try:
    client = QdrantClient(url="http://localhost:6333")
    info = client.get_collections()
    print("✅ Qdrant: Connected (Running native binary)")
except Exception as e:
    print(f"❌ Qdrant: Failed - {e}")
```

3. Check Redis

```
try:
    import redis
    r = redis.Redis(host='localhost', port=6379, db=0)
    r.ping()
    print("✅ Redis: Connected")
except Exception as e:
    print(f"❌ Redis: Failed - {e}")
```

```
if __name__ == "__main__":
    check_services()
```

Run the check:

Bash

```
source venv/bin/activate
python check_stack.py
```

... [LangGraph + CrewAI: Crash Course for Beginners \[Source Code Included\]](#) ...

This updated guide integrates the **Black Dashboard React** template into your Agentic AI architecture.

Architectural Shift: Instead of a generic Next.js app, we will use **Black Dashboard React** (a `create-react-app` based template) as the frontend. This requires us to configure a **Proxy** so the React Dashboard (Port 3000) can talk to your Node.js Backend-for-Frontend (Port 5000), which in turn talks to the Python Agents (Port 8000).

Step 1: Download & Setup Black Dashboard React

We will replace the generic `frontend` folder from the previous steps with this template.

1.1 Download the Template You can download it directly or clone the repository.

```
Bash
cd ~/agentic-ai
# Remove the previous generic frontend if you created it
rm -rf frontend

# Clone the official repo
git clone https://github.com/creativetimofficial/black-dashboard-react.git frontend
```

1.2 Install Dependencies This template relies on `node-sass` and older `react-scripts` which can conflict with Node 20. We will use the `--legacy-peer-deps` flag to ensure smooth installation.

```
Bash
cd frontend

# Install dependencies ignoring version conflicts
npm install --legacy-peer-deps

# Fix for Node 17+ (OpenSSL issue)
# Add this to your ~/.bashrc or run it in your current terminal session
export NODE_OPTIONS=--openssl-legacy-provider
```

1.3 Test the Dashboard Start the development server to ensure the UI loads.

```
Bash
npm start
```

- **Verify:** Open <http://localhost:3000>. You should see the dark-themed dashboard.
-

Step 2: Configure the Node.js BFF (Backend)

We need a Node.js server to act as the bridge between your React Dashboard and the Python AI agents.

2.1 Create the BFF Service

```
Bash
cd ~/agentic-ai
mkdir -p node-bff
cd node-bff
npm init -y
npm install express cors dotenv socket.io-client
```

2.2 Create the Server File Create [~/agentic-ai/node-bff/index.js](#):

```
JavaScript
const express = require('express');
const cors = require('cors');
const { io } = require('socket.io-client');
const app = express();

app.use(cors());
app.use(express.json());

// Connect to Python Agent Service (WebSocket)
const pythonSocket = io('ws://localhost:8000', {
  reconnection: true
});

pythonSocket.on('connect', () => {
  console.log('✅ Connected to Python Agent Service');
});

// API Endpoint for React Frontend to trigger agents
app.post('/api/chat', (req, res) => {
```



```

const { message } = req.body;

// Forward task to Python Agents
console.log(`Forwarding to agents: ${message}`);
pythonSocket.emit('user_message', message);

// In a real app, you would handle the streaming response here
// For now, we confirm receipt
res.json({ status: 'Processing', message: 'Agent started' });
});

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`🚀 Node BFF running on port ${PORT}`);
});

```

Step 3: Connect React Dashboard to Node BFF

Now we modify the Black Dashboard to talk to our Node server instead of using mock data.

3.1 Configure Proxy Edit `~/agentic-ai/frontend/package.json`. Add the `proxy` line so API requests go to port 5000.

```

JSON
{
  "name": "black-dashboard-react",
  "version": "1.2.2",
  "proxy": "http://localhost:5000",
  "dependencies": { ... }
}

```

3.2 Create the AI Chat Component We will create a new view in the dashboard for interacting with the agents. Create `~/agentic-ai/frontend/src/views/AgentChat.js`:

```

JavaScript
import React, { useState } from "react";
// reactstrap components
import {
  Button,
  Card,
  CardHeader,
  CardBody,

```

```

    FormGroup,
    Form,
    Input,
    Row,
    Col,
  } from "reactstrap";

function AgentChat() {
  const [input, setInput] = useState("");
  const [response, setResponse] = useState("Agent is ready...");

  const handleSubmit = async (e) => {
    e.preventDefault();
    setResponse("Thinking...");

    try {
      // Call our Node.js BFF
      const res = await fetch('/api/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: input })
      });
      const data = await res.json();
      setResponse(data.message); // In real implementation, this would connect to a WebSocket
    } catch (err) {
      setResponse("Error connecting to server.");
    }
  };

  return (
    <div className="content">
      <Row>
        <Col md="12">
          <Card>
            <CardHeader>
              <h5 className="title">Agentic AI Console</h5>
            </CardHeader>
            <CardBody>
              <div style={{ minHeight: "200px", padding: "20px", background: "#1e1e2f", border: "1px solid #2b3553", borderRadius: "5px", marginBottom: "20px", color: "#fff" }}>
                {response}
              </div>
              <Form onSubmit={handleSubmit}>

```

```

    <Row>
      <Col md="10">
        <FormGroup>
          <Input
            placeholder="Ask the agents..."
            type="text"
            value={input}
            onChange={(e) => setInput(e.target.value)}
            style={{ color: "#fff" }}
          />
        </FormGroup>
      </Col>
      <Col md="2">
        <Button className="btn-fill" color="primary" type="submit">
          Send
        </Button>
      </Col>
    </Row>
  </Form>
</CardBody>
</Card>
</Col>
</Row>
</div>
);
}

```

```
export default AgentChat;
```

3.3 Register the Route Edit `~/agentic-ai/frontend/src/routes.js` to add your new page to the sidebar.

JavaScript

```
import AgentChat from "views/AgentChat.js";
```

```

var routes = [
  {
    path: "/agent-chat",
    name: "AI Agents",
    icon: "tim-icons icon-atom",
    component: AgentChat,
    layout: "/admin",
  },

```

```
// ... keep existing routes  
];  
export default routes;
```

Step 4: Full System Startup

You now have 3 terminal windows to manage:

Terminal 1: Infrastructure (Qdrant & Ollama)

```
Bash  
# Ensure Qdrant is running  
./qdrant
```

```
# Ensure Ollama is ready  
ollama serve
```

Terminal 2: Python Backend (Agents)

```
Bash  
cd ~/agentic-ai/backend  
source venv/bin/activate  
# Run your Python FastAPI/Socket server (assuming you created main.py)  
uvicorn main:app --reload --port 8000
```

Terminal 3: Node.js BFF

```
Bash  
cd ~/agentic-ai/node-bff  
node index.js
```

Terminal 4: React Dashboard

```
Bash  
cd ~/agentic-ai/frontend  
npm start
```

Summary of Architecture Flow

1. User types into the **Black Dashboard (Port 3000)**.

2. Request is proxied to **Node.js BFF (Port 5000)**.
3. Node.js forwards the prompt via WebSocket to **Python (Port 8000)**.
4. Python spins up a **LangGraph** workflow.
5. Agents query **Qdrant** and generate answers using **Ollama**.
6. Response flows back up the chain to the UI.

Relevant Resource

... [Black Dashboard React - Official Documentation](#) ...

This video provides a visual walkthrough of the template's structure, which is helpful since the folder layout of Creative Tim templates can differ slightly from standard React apps.

WINDOWS

This guide outlines the steps to build the Agentic AI environment natively on **Windows 10/11** without Docker or WSL.

Prerequisites

- **PowerShell** or **Command Prompt** (Run as Administrator for installations)
- **Git for Windows**: [Download Here](#)
- **Visual Studio Build Tools** (Select "Desktop development with C++" during install - required for some Python vector libraries): [Download Here](#)

Step 1: Install & Run Qdrant (Native Windows)

Qdrant provides a standalone executable for Windows.

1. Go to the [Qdrant Releases Page](#).
2. Look for the latest version (e.g., **v1.11.x**) and download the file ending in **...-x86_64-pc-windows-msvc.zip**.
3. Extract the ZIP file to a folder, e.g., **C:\qdrant**.

Open PowerShell, navigate to the folder, and run the executable:

```
PowerShell
cd C:\qdrant
.\qdrant.exe
```

4.
 - **Verify**: Open your browser to <http://localhost:6333/dashboard>.
 - *Keep this PowerShell window open.*

Step 2: Install & Run Ollama (Windows)

1. Download the **Ollama Windows Installer** from ollama.com.
2. Run **OllamaSetup.exe**. It will install and automatically start in your system tray.

Open a **new** PowerShell window and pull the Mistral model:

PowerShell

ollama pull mistral

3.
 - **Verify:** `ollama run mistral "Hello AI"`

Step 3: Node.js & React Frontend (Black Dashboard)

We will use **Black Dashboard React** as the UI.

3.1 Install Node.js

1. Download the **LTS Version** (e.g., v20.x) **.msi** installer from nodejs.org.
2. Install it and ensure "Add to PATH" is checked.
3. Restart PowerShell to confirm: `node -v`

3.2 Setup Black Dashboard

PowerShell

Create project root

`mkdir C:\AgenticAI`

`cd C:\AgenticAI`

Clone the dashboard

`git clone https://github.com/creativetimofficial/black-dashboard-react.git frontend`

Enter directory

`cd frontend`

Install dependencies (fix for older dependencies in the template)

`$env:NODE_OPTIONS="--openssl-legacy-provider"`

`npm install --legacy-peer-deps`

3.3 Configure Proxy Open `C:\AgenticAI\frontend\package.json` in Notepad or VS Code and add the proxy line so React can talk to your backend:

```
JSON
{
  "name": "black-dashboard-react",
  "proxy": "http://localhost:5000",
  "dependencies": { ... }
}
```

Step 4: Python Backend (LangGraph + CrewAI)

4.1 Install Python

1. Download Python 3.12 installer from python.org.
2. **Crucial:** Check the box **"Add Python to PATH"** before clicking Install.

4.2 Setup Environment

```
PowerShell
cd C:\AgenticAI
mkdir backend
cd backend
```

```
# Create Virtual Environment
python -m venv venv
```

```
# Activate Environment
.\venv\Scripts\Activate.ps1
```

Note: If you get a permission error, run: `Set-ExecutionPolicy Unrestricted -Scope Process`

4.3 Install Libraries We will use **SQLite** for persistence instead of Redis to keep it native to Windows.

```
PowerShell
pip install langgraph crewai crewai[tools] langchain langchain-community langchain-qdrant
langchain-ollama fastapi uvicorn websockets python-socketio
```

```
# Install SQLite checkpointer specifically for LangGraph
pip install langgraph-checkpoint-sqlite
```

4.4 Create Backend Code Create a file `C:\AgenticAI\backend\main.py` using your code editor:

```
Python
from fastapi import FastAPI
import socketio
from langchain_ollama import ChatOllama
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3

# 1. Setup FastAPI & Socket.IO
app = FastAPI()
sio = socketio.AsyncServer(async_mode='asgi', cors_allowed_origins='*')
app.mount("/", socketio.ASGIApp(sio))

# 2. Setup Persistence (Native SQLite file)
conn = sqlite3.connect("checkpoints.sqlite", check_same_thread=False)
memory = SqliteSaver(conn)

# 3. Define Simple Agent Logic (Placeholder)
llm = ChatOllama(model="mistral")

@sio.event
async def user_message(sid, data):
    print(f"Received from UI: {data}")

    # Simple direct LLM call for testing
    # (Replace this with your full LangGraph/CrewAI workflow later)
    response = llm.invoke(data)

    await sio.emit('agent_response', {"content": response.content}, room=sid)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Step 5: Node.js Intermediate Server (BFF)

This server sits between React and Python to handle WebSocket traffic cleanly.

```
PowerShell
cd C:\AgenticAI
```



```
mkdir node-bff
cd node-bff
npm init -y
npm install express cors socket.io-client dotenv
```

Create `C:\AgenticAI\node-bff\index.js`:

JavaScript

```
const express = require('express');
const cors = require('cors');
const { io } = require('socket.io-client');
const app = express();

app.use(cors());
app.use(express.json());

// Connect to Python Backend (Port 8000)
const pythonSocket = io('ws://localhost:8000', {
  transports: ['websocket']
});

pythonSocket.on('connect', () => console.log('✅ Connected to Python Agents'));
pythonSocket.on('agent_response', (data) => console.log('Agent says:', data));

// API for React
app.post('/api/chat', (req, res) => {
  const { message } = req.body;
  pythonSocket.emit('user_message', message);
  res.json({ status: 'Sent to Agent' });
});

app.listen(5000, () => console.log('🚀 Node BFF on Port 5000'));
```

Step 6: Run Everything

You will need **4 PowerShell Windows** open at once:

1. **Qdrant**: `.\qdrant.exe` (Inside `C:\qdrant`)

Backend:

```
PowerShell
cd C:\AgenticAI\backend
```

```
.\venv\Scripts\activate
uvicorn main:app --reload --port 8000
```

2.

Node BFF:

```
PowerShell
cd C:\AgenticAI\node-bff
node index.js
```

3.

Frontend:

```
PowerShell
cd C:\AgenticAI\frontend
$env:NODE_OPTIONS="--openssl-legacy-provider"
npm start
```

4.

Final Test: Open <http://localhost:3000>. The requests will flow: **React (3000) -> Node BFF (5000) -> Python/LangGraph (8000) -> Qdrant/Ollama.**